Lecture 16

Neural Network Design and Regularization

Parameter Initialization, Normalization, Double Descent, Model Averaging, Drop-out, and Residual Connections.

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi

What would you like to learn about Deep Learning or PyTorch?

How do you feel about the midterm. (1 = Not Great, and 5 = Wonderful)

Join at slido.com
#8111828

Parameter Initialization

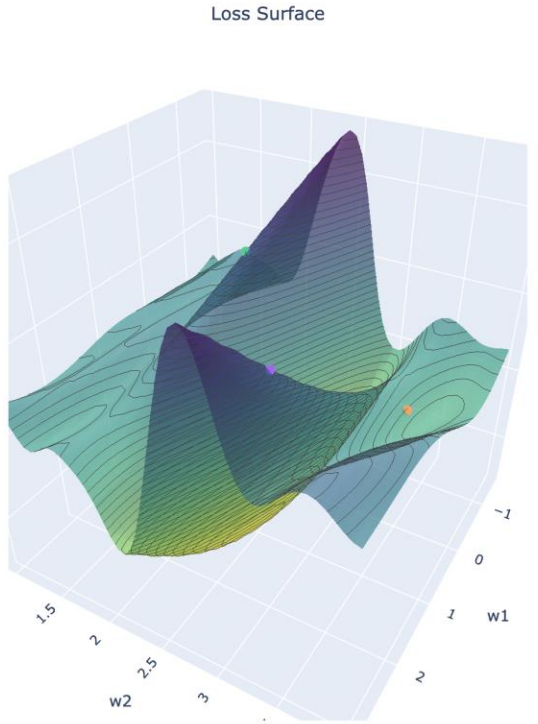
Choice of Initial Parameters Matters

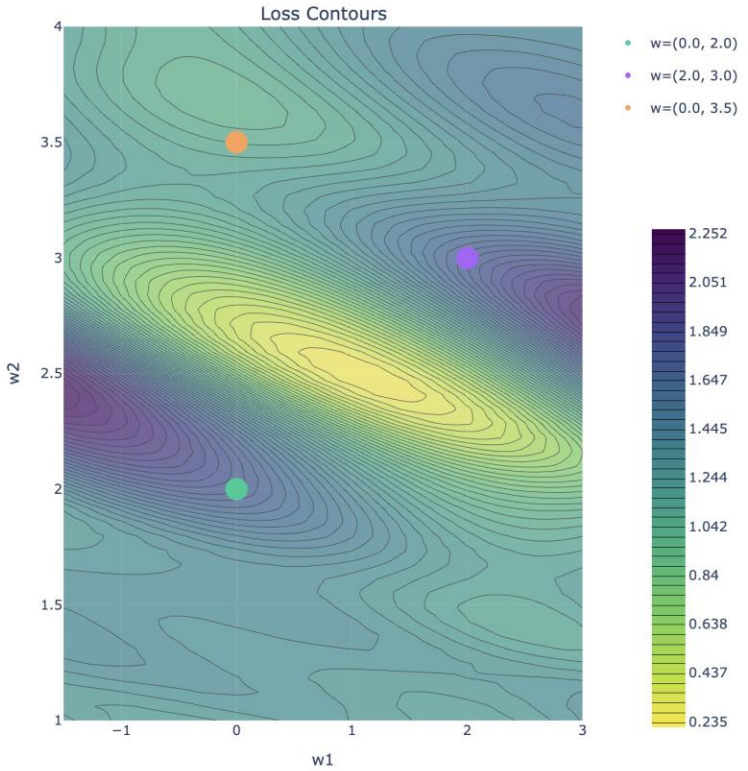
8111828

The neural network loss surface is **highly non-convex** so choice of starting point can result in:

- diverging solutions
- slow convergence
- bad local minima

What makes a good starting point?



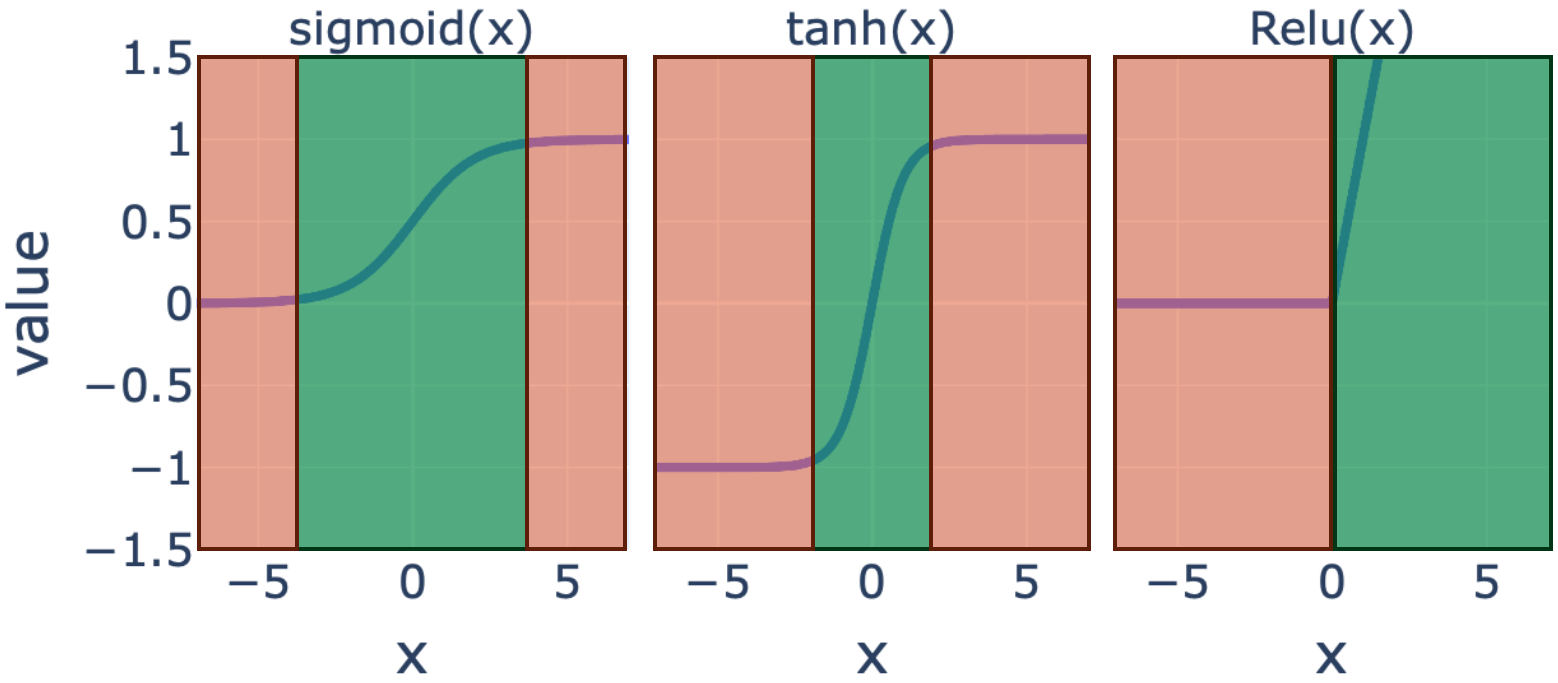


“Sweet-spot” of the Non-Linearity

8111828

Sweet-spot is near the **transition point** in the non-linear transformation.

- Non-zero gradient
- Typically **near zero**



Outside the transition point the activation is **saturated**.

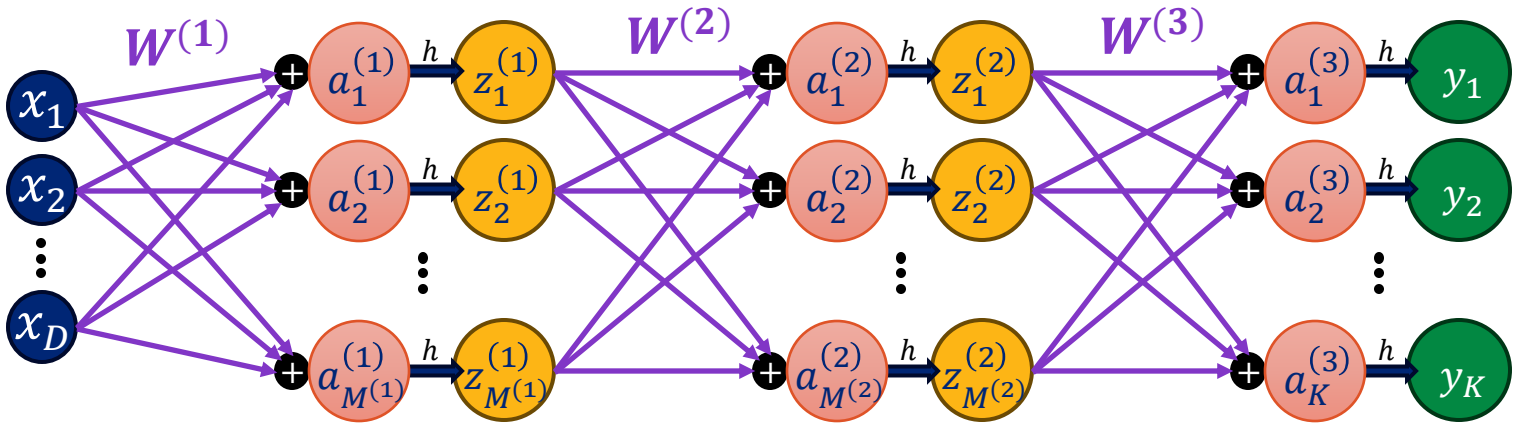
- Small changes don’t change activation.

Starting at Zero-Weights

8111828

If we initialize all weights to zero, the activations will all be zero (at the **transition point**).

Example: Two hidden-layer neural network



$$a^{(1)} = W^{(1)}x \qquad z^{(1)} = h\left(a^{(1)}\right)$$
$$a^{(2)} = W^{(2)}z^{(1)} \qquad z^{(2)} = h\left(a^{(2)}\right)$$
$$a^{(3)} = W^{(3)}z^{(2)} \qquad y = h\left(a^{(3)}\right)$$

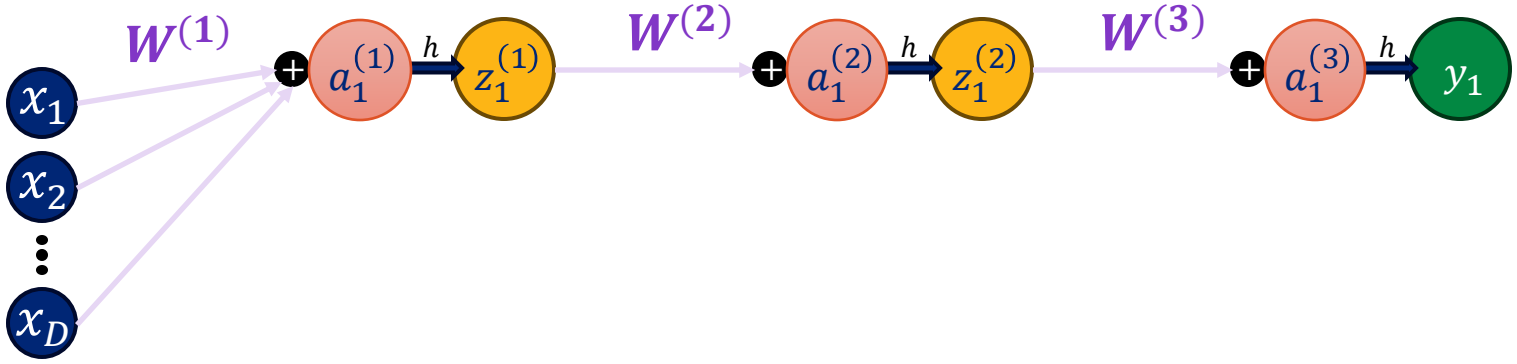
$$y = h\left(W^{(3)}h\left(W^{(2)}\,h\left(W^{(1)}x\right)\right)\right)$$

- If all $W^{(l)} = 0$ then all the pre-activations $a^{(l)} = 0$ are also zero.

Why shouldn't we initialize all our neural network's weights to all zero?

Unbroken Symmetry at Zero-Weights

If we initialize all weights to zero, then all hidden units produce **identical activations** and **remain the same during training**.



If we use a **relu** or **tanh** activation the gradient will be zero.

$$\frac{\partial L(t, y(x; w))}{\partial W_{km}^{(3)}} = L'(t, y(x; w)) \left(\frac{\partial y(x; w)}{\partial W_{km}^{(3)}} \right) = L'(t, y(x; w))\, h'\left(a^{(3)}\right) \underbrace{h\left(a^{(2)}\right)}_{0}{}_{km}$$

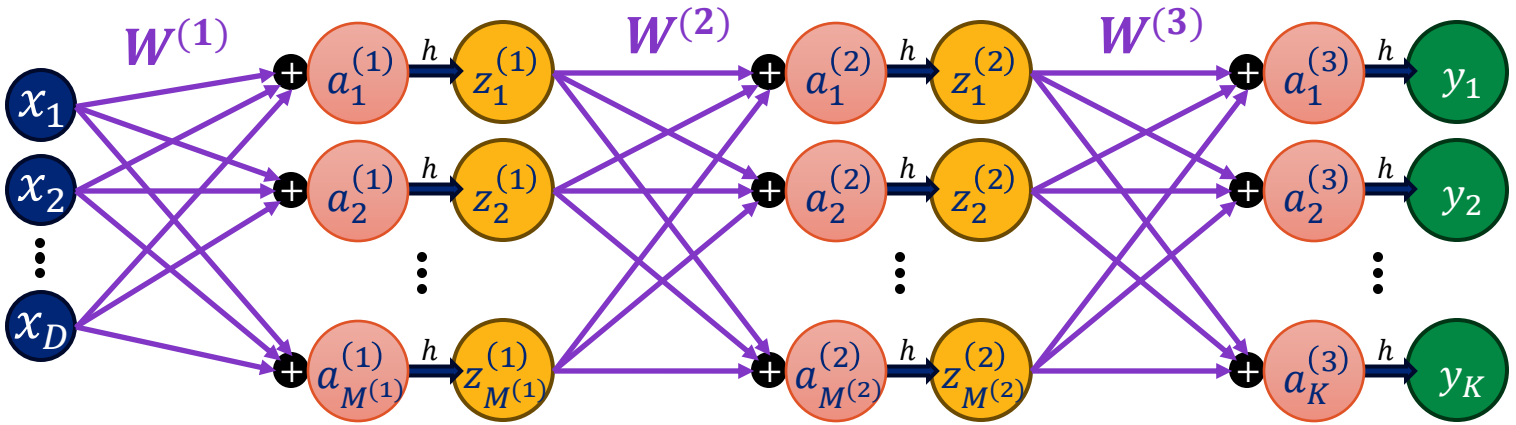
- **No Learning!**

Randomization is needed to **break symmetry** across neurons.

Weight Symmetry In Neural Networks

8111828

For any layer in the neural network, we could “re-arrange” the neurons to obtain an identical network.

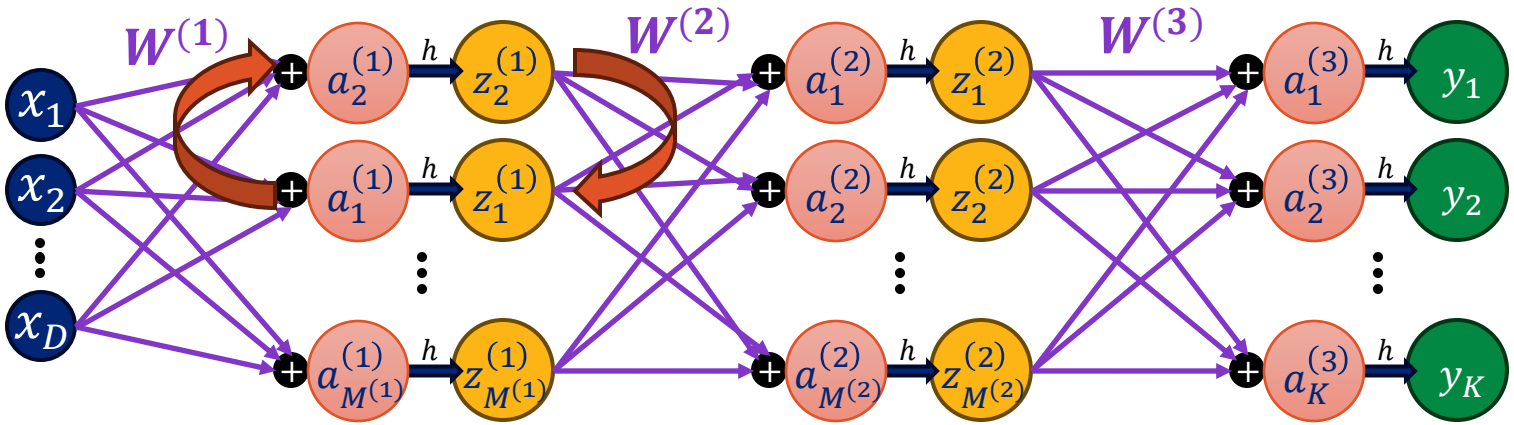


Bishop 6.2.4

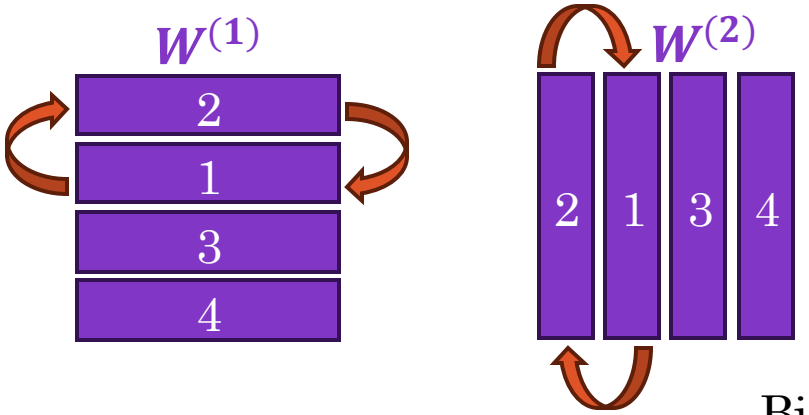
Weight Symmetry In Neural Networks

8111828

For any layer in the neural network, we could "re-arrange" the neurons to obtain an identical network.



- This corresponds to **permuting rows** and **columns** of the weight matrices
- For each layer l we can permute the $M^{(l)}$ **neurons** $\boldsymbol{M^{(l)}!}$ **ways**.

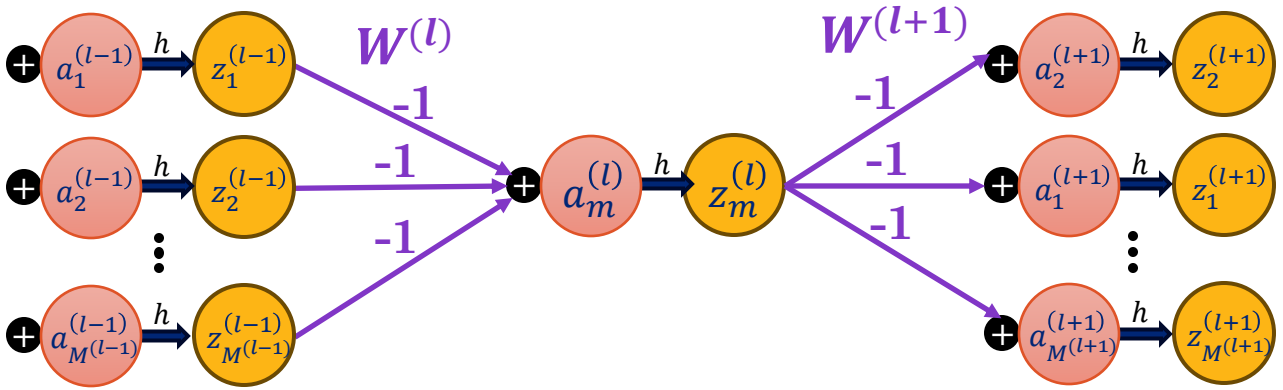


Bishop 6.2.4

Sign Flipping Equivalence in Networks

8111828

For **odd activation functions** where $h(-a) = -h(a)$ (e.g., tanh), we can **negate the weights** to produce equivalent networks:



$$a^{(l+1)} = W^{(l+1)}\, h\left(W^{(l)} z^{(l-1)}\right)$$
$$= \left(-W^{(l+1)}\right) h\left(\left(-W^{(l)}\right) z^{(l-1)}\right)$$

- This can be done for individual neurons corresponding to rows of $-W^{(l)}$ and columns of $-W^{(l+1)}$
- Sign flipping produces an additional $\mathbf{2^{M^{(l)}}}$ **equivalent networks** for layer l.

Bishop 6.2.4

Counting Network Symmetries

8111828

Summarizing the two primary symmetries for each layer l with $M^{(l)}$ **neurons**:

- **Permutations:** $M^{(l)}!$
- **Sign Flipping:** $2^{M^{(l)}}$

Therefore, a **single parameterization** has MANY equivalent networks:

$$\prod_{l=1}^{L}\left(M^{(l)}!\right)\left(2^{M^{(l)}}\right)$$

Is this an issue?

- No, we just need to find a "good parameterization"
- The **good parameterizations** have **many equivalent forms**

Breaking Symmetry with Random Weight Initialization

8111828

The primary goal of weight initialization is to **break weight symmetry**

Weights are **randomly initialized** by sampling from a distribution:

- **Uniform**$[-\epsilon, \epsilon]$: provides robust starting point (most common)
- $\mathcal{N}(0, \epsilon^2)$: occasionally larger outliers for greater symmetry breaking.

Biases are typically initialized at zero.

- No need to break symmetry for the bias term.

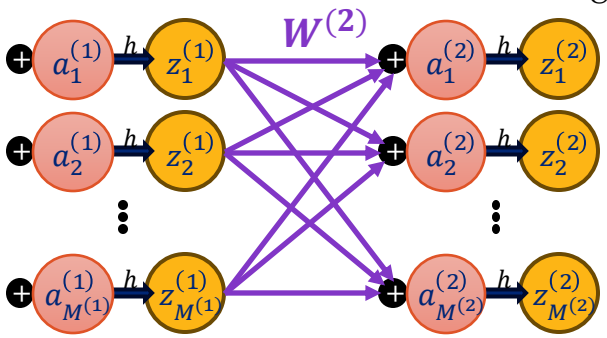
The **choice of ϵ is important** as we want to ensure that the activations (and gradients) don't become too large (**explode**) or too small (**vanish**) across layers.

- **Mathematically:** keep the **variance constant** across layers.

He Initialization for ReLU Activations

8111828

Consider the logits $a_m^{(l)}$ and activations $z_m^{(l)}$ on layer l in the network:



$$a_m^{(l)} = \sum_{i=1}^{M^{(l-1)}} W_{mi}^{(l)} z_i^{(l-1)}; \qquad z_m^{(l)} = \mathrm{ReLU}\left(a_m^{(l)}\right)$$

If we assume $z_i^{(l-1)}$ and $W_{mi}^{(l)}$ **are IID** with $\mathrm{Var}\left[z_i^{(l-1)}\right] = \lambda^2$ and $\mathrm{Var}\left[W_{mi}^{(l)}\right] = \epsilon^2$ then:

$$\mathrm{Var}\left[z_m^{(l)}\right] = \frac{1}{2}\sum_{i=1}^{M^{(l-1)}} \mathrm{Var}\left[W_{mi}^{(l)}\right]\mathrm{Var}\left[z_i^{(l-1)}\right] = \frac{1}{2}\sum_{i=1}^{M^{(l-1)}} \epsilon^2\lambda^2 = \frac{M^{(l-1)}}{2}\epsilon^2\lambda^2$$

Since we want to preserve the variance across layers:

$$\frac{M^{(l-1)}}{2}\epsilon^2\lambda^2 = \lambda^2 \quad \Rightarrow \quad \epsilon = \sqrt{\frac{2}{M^{(l-1)}}}$$

He initialization is then $W_{mi}^{(l)} \sim \mathcal{N}(0, \frac{2}{M^{(l-1)}})$ or $W_{mi}^{(l)} \sim$ **Uniform** $\left[-\sqrt{\frac{6}{M^{(l-1)}}}, \sqrt{\frac{6}{M^{(l-1)}}}\right]$

Bishop 7.2.5

Xavier Initialization for Tanh Activations

8111828

Follows a similar argument to He initialization but addressing backward vanishing gradients for **tanh activations**:

$$\epsilon^2 = \frac{2}{M^{(l)} + M^{(l-1)}}$$

Fan-out

Fan-in

Xavier initialization is accomplished by sampling weights according to:

$$W_{mi}^{(l)} \sim \mathcal{N}\left(0, \frac{2}{M^{(l)} + M^{(l-1)}}\right)$$

$$W_{mi}^{(l)} \sim \textbf{Uniform}\left[-\sqrt{\frac{6}{M^{(l)} + M^{(l-1)}}}, \sqrt{\frac{6}{M^{(l)} + M^{(l-1)}}}\right]$$

Bishop 7.2.5

Weight Initialization in PyTorch

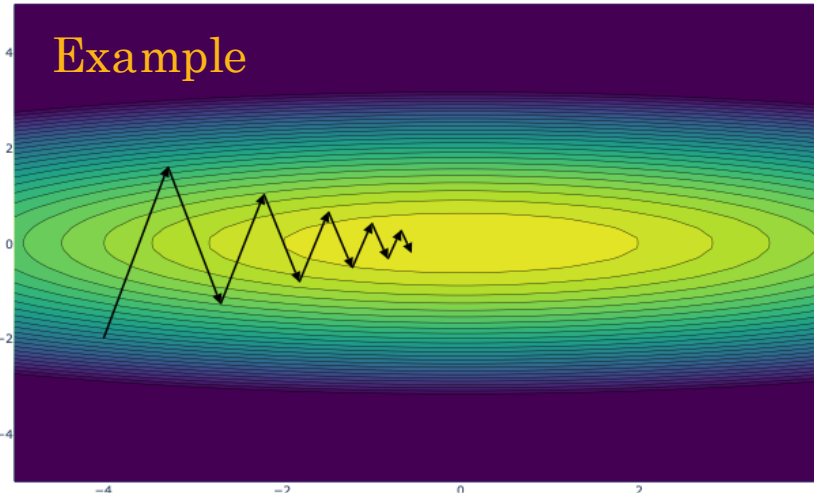
```python
class MLPModel(nn.Module):
    def __init__(self, dims, activation="relu"):
        super().__init__()
        self.activation = activation
        self.layers = nn.ModuleList(
            [nn.Linear(dims[i], dims[i+1]) for i in range(len(dims)-1)]
        )
        self.act = nn.functional.relu if activation == "relu" else torch.tanh
        self._reset_parameters()

    def _reset_parameters(self):
        # Initialize weights according to activation function
        for i, layer in enumerate(self.layers):
            if not isinstance(layer, nn.Linear):
                continue
            if self.activation == "relu":
                # He/Kaiming for ReLU
                nn.init.kaiming_uniform_(layer.weight, nonlinearity="relu")
            else:
                # Xavier for tanh
                nn.init.xavier_uniform_(layer.weight)
            nn.init.zeros_(layer.bias)

    def forward(self, x):
        for i, layer in enumerate(self.layers):
            x = layer(x)
            if i < len(self.layers) - 1: # no activation on last layer
                x = self.act(x)
        return x
```

Normalization

Data Normalization

8111828

Earlier we saw that inputs to a network (or layer) that vary significantly in scale can lead to slow convergence.



One solution is **feature standardization**:

$$\hat{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}$$

where μ_i and σ_i are computed from the training data:

$$\mu_i = \frac{1}{N}\sum_{n=1}^{N} x_{ni} \qquad \sigma_i^2 = \frac{1}{N}\sum_{n=1}^{N} (x_{ni} - \mu_i)^2$$

Then, at test-time we use the same μ_i and σ_i.

Bishop 7.4.2

Normalization in the Network

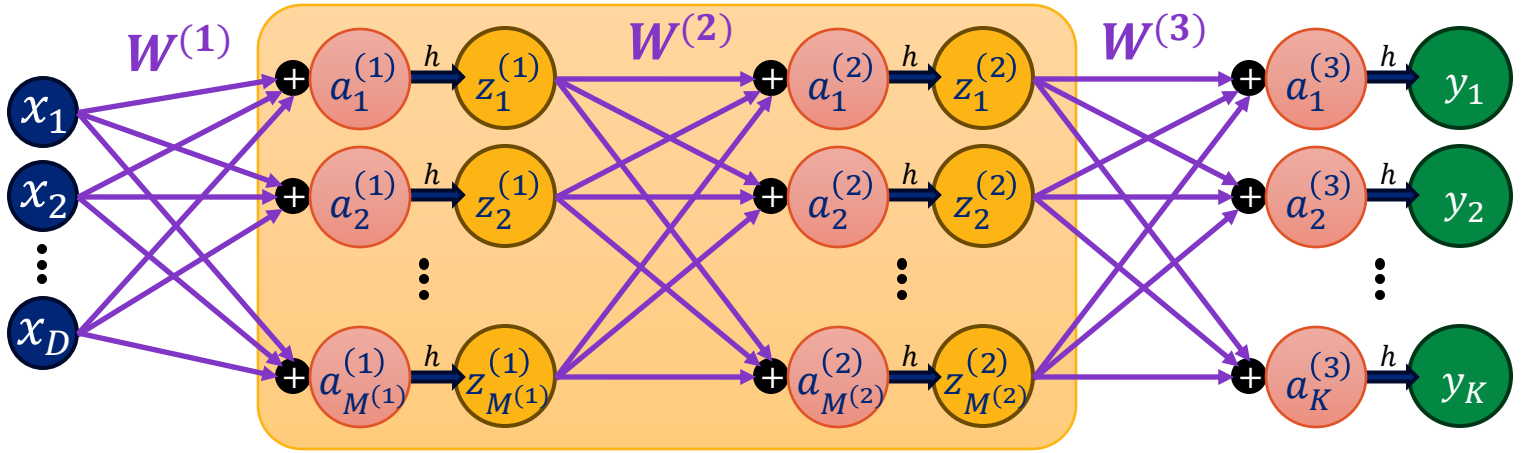
8111828

Could we apply normalization to interior layers of the network?

Example:

- 2-hidden layer NN

$$y = h\left(W^{(3)} h\left(W^{(2)}\, h\left(W^{(1)} x\right)\right)\right)$$



Why would **hidden layer normalization** help?

- Ensure internal activations are similar scale
  - Chain rule → gradients stay similar scale

Problem: internal activations change during learning (SGD)!

Bishop 7.4.2

Batch Normalization

8111828

Apply **normalization** (standardization) to **activations** $\hat{z}_{km}^{(l)}$ during learning by computing μ_i and σ_i within a **mini-batch** of SGD.

$$\hat{z}_{km}^{(l)} = \frac{z_{km}^{(l)} - \mu_m^{(l)}}{\sqrt{\left(\sigma_m^{(l)}\right)^2 + \delta}}$$

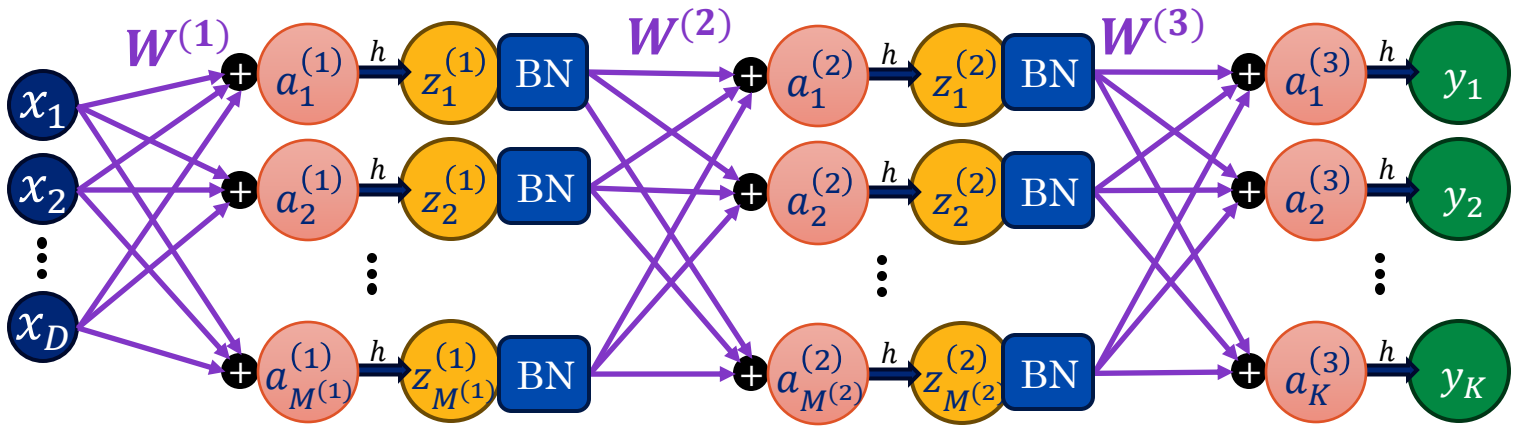
Activation m on layer l for the k^{th} datapoint in the mini-batch

$$\mu_m^{(l)} = \frac{1}{K}\sum_{k=1}^{K} z_{km}^{(l)}$$

Mean over the mini-batch of K datapoints

$$\left(\sigma_m^{(l)}\right)^2 = \frac{1}{K}\sum_{k=1}^{K}\left(z_{km}^{(l)} - \mu_m^{(l)}\right)^2$$

Variance over the mini-batch of K datapoints



Could also be applied to **pre-activations** $a_{nm}^{(l)}$.

Making Predictions with Batch Norm.

8111828

To make a prediction at test time, we need the mean $\mu_m^{(l)}$ and variance $\left(\sigma_m^{(l)}\right)^2$ of the hidden layer activations from training.

Ideally, we would compute them from the entire training dataset.

- Expensive

Solution – Compute running exponential average:

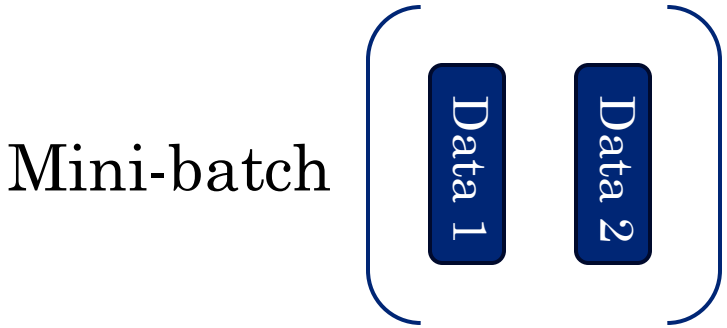
$$\mu_m^{(l,\tau)} = \alpha\left(\mu_m^{(l,\tau-1)}\right) + (1-\alpha)\frac{1}{K}\sum_{k=1}^{K} z_{km}^{(l)}$$

$$\sigma_m^{(l,\tau)} = \alpha\left(\sigma_m^{(l,\tau-1)}\right) + (1-\alpha)\sqrt{\frac{1}{K}\sum_{k=1}^{K}\left(z_{km}^{(l)} - \mu_m^{(l)}\right)^2}$$

Batch Normalization is Difficult to Parallelize

8111828

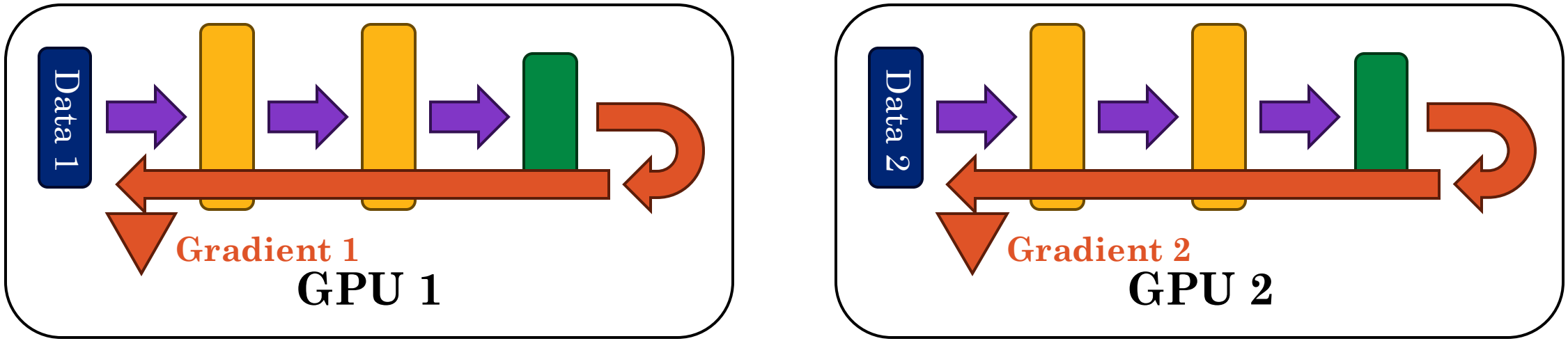
Data-parallel stochastic gradient descent



Batch Normalization is Difficult to Parallelize

8111828

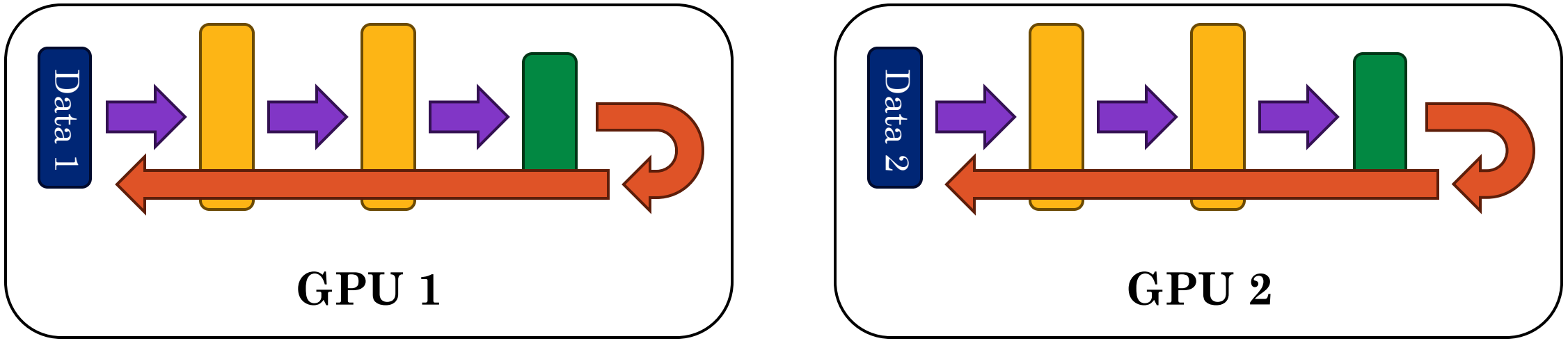
Data-parallel stochastic gradient descent



Batch Normalization is Difficult to Parallelize

8111828

Data-parallel stochastic gradient descent



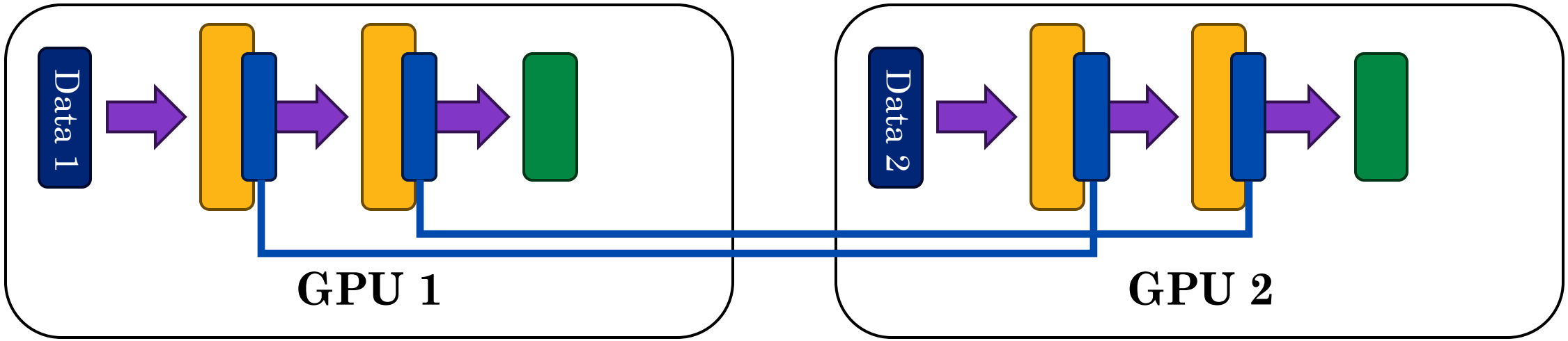
GPUs share gradients and compute sum (called **all-reduce**).

Gradient 1 + Gradient 2

Batch Normalization is Difficult to Parallelize

8111828

Batch normalization **couples the layers** in the forward pass.



Cannot compute gradients **independently** across GPUs

Layer Normalization

8111828

Layer Normalization computes the layer mean and variance across the **neurons in the layer** instead of across the mini-batch:

$$\hat{z}_{km}^{(l)} = \frac{z_{km}^{(l)} - \mu_k^{(l)}}{\sqrt{\left(\sigma_k^{(l)}\right)^2 + \delta}}$$

Activation m on layer l for the k^{th} datapoint in the mini-batch

$$\mu_k^{(l)} = \frac{1}{M^{(l)}} \sum_{m=1}^{M^{(l)}} z_{km}^{(l)}$$

Mean over the $M^{(l)}$ hidden activations

$$\left(\sigma_k^{(l)}\right)^2 = \frac{1}{M^{(l)}} \sum_{m=1}^{M^{(l)}} \left(z_{km}^{(l)} - \mu_m^{(l)}\right)^2$$

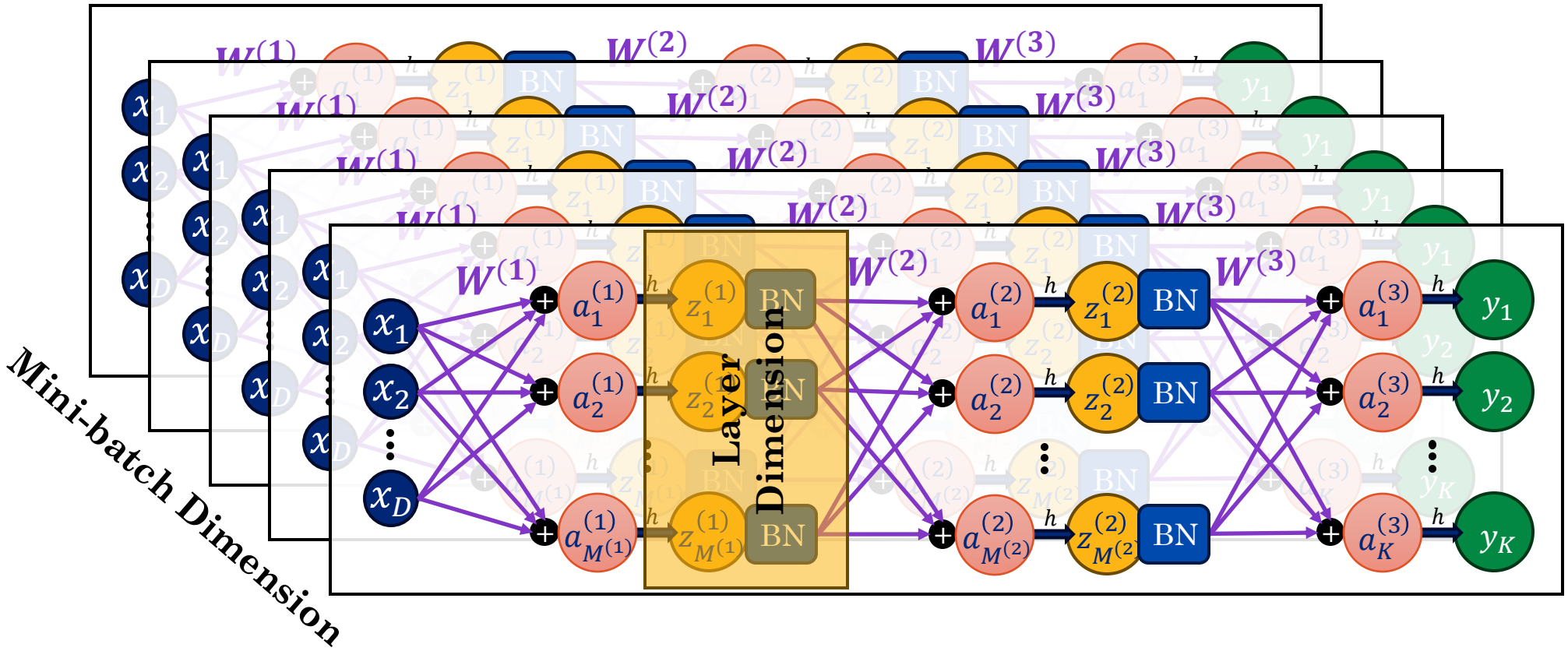
Variance over over the $M^{(l)}$ hidden activations

Normalization is applied the same way at test-time.

- No need to estimate on training data.

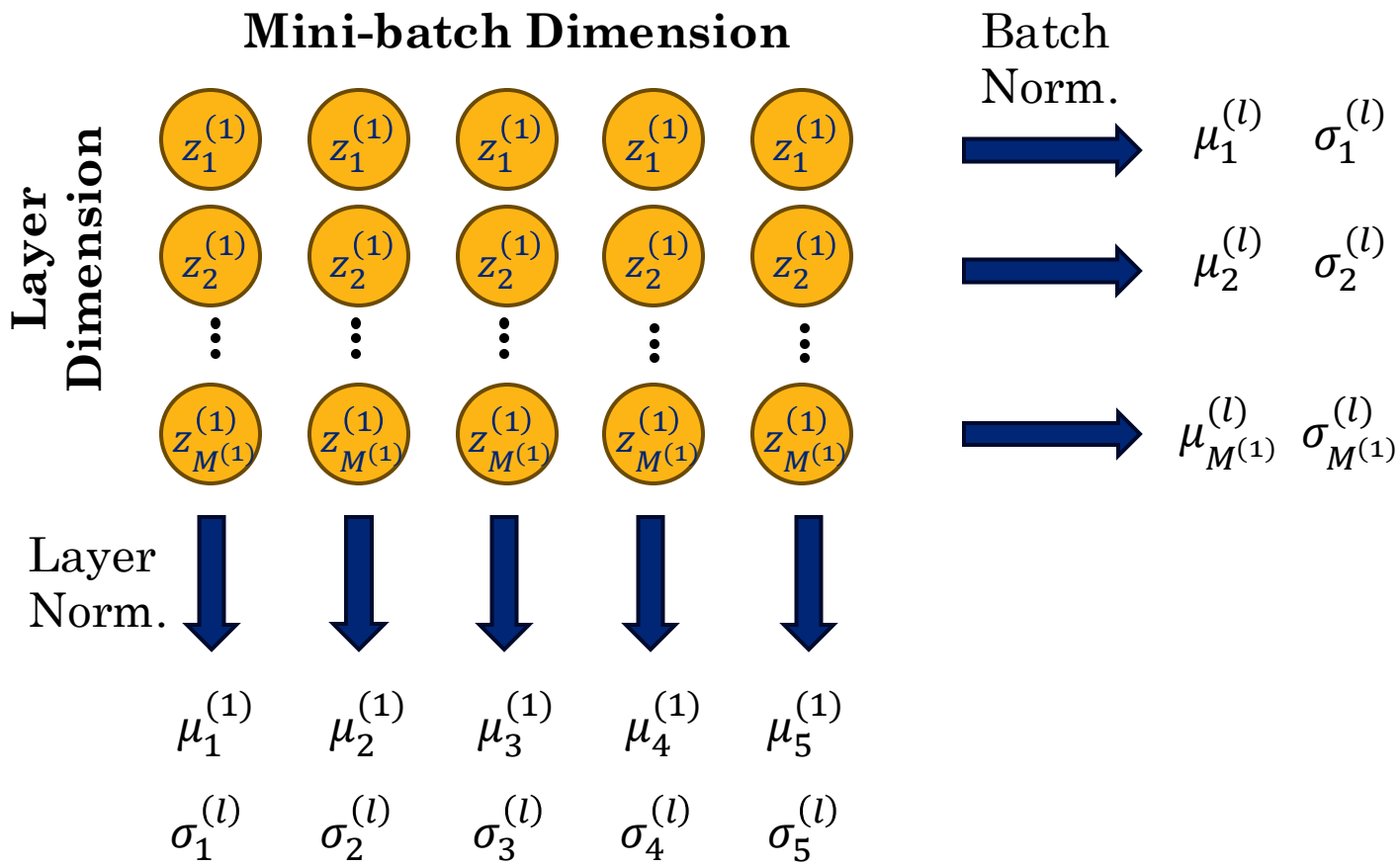
Layer Normalization is currently **used in large-language models**.

Layer Normalization vs Batch Normalization



Layer Normalization vs Batch Normalization

8111828



Re-Scaling with Normalization

8111828

Normalization limits the **degrees of freedom** of a layer.

- For example, by centering the distribution it is no longer possible for all activations to be positive.

Solution: Learned scale $\gamma_m^{(l)}$ and shift $\beta_m^{(l)}$ parameters

$$\tilde{z}_{km}^{(l)} = \gamma_m^{(l)} \hat{z}_{km}^{(l)} + \beta_m^{(l)}$$

The parameters $\gamma_m^{(l)}$ and $\beta_m^{(l)}$ are learned through SGD.

- This generally improves convergence.

Batch-Norm and Layer-Norm in PyTorch

8111828

Creating norm layers:

```
if norm_kind == "batch":
    self.norms = nn.ModuleList([nn.BatchNorm1d(c) for c in dims[1:]])
else: # norm_kind == "layer":
    self.norms = nn.ModuleList([nn.LayerNorm(c) for c in dims[1:]])
```

Using norm layers:

```
def forward(self, x):
    for i, (layer, norm) in enumerate(zip(self.layers, self.norms)):
        x = layer(x)
        if i < len(self.layers) - 1:
            x = norm(x)
            x = self.act(x)
    return x
```

Inductive Biases

Learning Requires Inductive Biases

8111828

"No Free Lunch Theorem" – any learning algorithm is as good as any other when **considering all possible problems**.

- Need to exploit **inductive biases.**

We **encode inductive biases** in neural networks through:

- **Pre-processing:** Transform the data to enforce desired invariances (e.g., normalize scale, orientation, or intensity).
- **Regularized objectives:** Penalize models that exhibit undesired properties during training.
- **Data augmentation:** Expand the training data with transformed examples to make invariances explicit (e.g., rotated images).
- **Architecture design:** Choose model structures that build in invariances (e.g., convolution layers for translation invariance).

Stopped Here

Pre-processing through Feature Engineering

8111828

Feature Engineering – the process of **manually constructing features** common in many classical machine learning techniques.

Examples:

- **Term Frequence – Inverse Document Frequency (TF-IDF)** augment document representations based on the commonality of words.
- **Histogram of Oriented Gradients (HOG)** was used in computer vision to represent images features.
- **Mel-frequency cepstral coefficients** (**MFCCs**) are (still) used to represent sound in for speech recognition systems.

Modern **deep learning** uses large neural networks as "feature" functions to learn features that are "invariant" to the prediction task.

- Inductive bias shifts to data selection, model, and training process.

Weight Decay Regularization

8111828

Previously discussed regularization as an additional penalty $\mathbf{E_w}[w]$:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \tilde{\mathbf{E}}[\mathbf{w}] = \arg\min_{\mathbf{w}} \mathbf{E_D}[\mathbf{w}, \mathcal{D}] + \lambda\, \mathbf{E_w}[\mathbf{w}]$$

where λ is the regularization hyper-parameter.

We introduced Ridge-Regression with the L^2 regularization penalty:

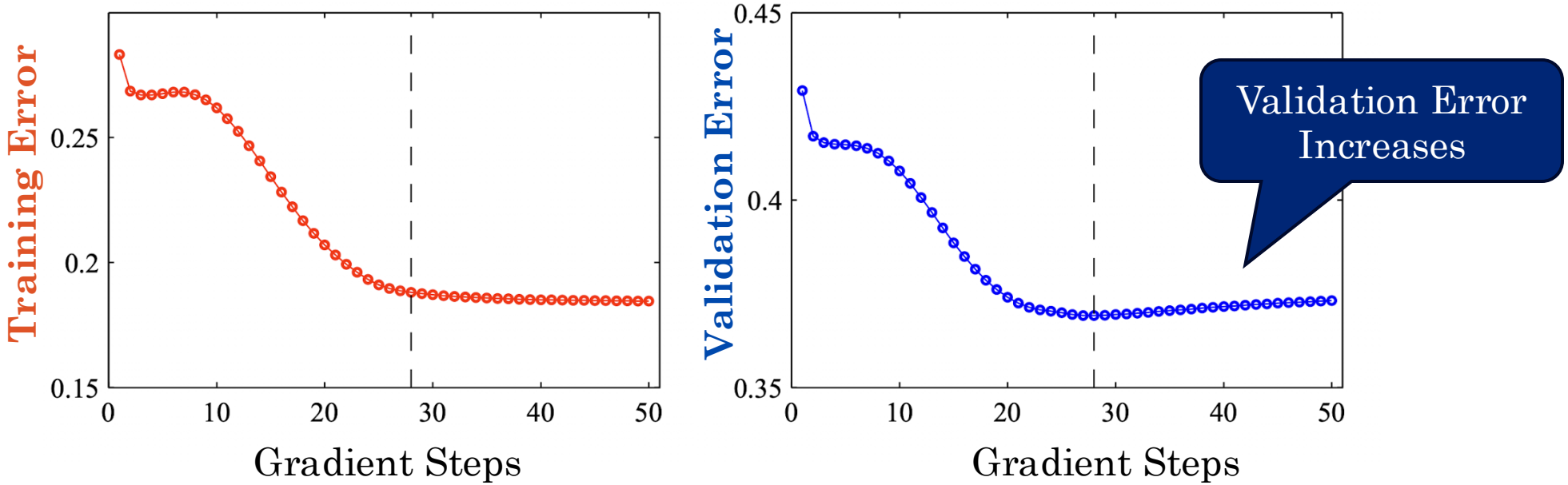
$$\mathbf{E_w}[\mathbf{w}] = \frac{1}{2}\sum_{m=1}^{M} \mathbf{w}_m^2 = \frac{1}{2}\mathbf{w}^T\mathbf{w}$$

In neural networks, this is called **weight decay** and is commonly applied in the gradient update:

$$\nabla\tilde{\mathbf{E}}[\mathbf{w}] = \nabla\mathbf{E_D}[\mathbf{w}] + \lambda\mathbf{w}$$

Learning Curves and Early Stopping

During training it is common practice to plot a **learning curve** which tracks the **training** and **validation error**



Early Stopping: stop training (return to the model checkpoint) when the validation error stopped decreasing.

Figure from Bishop Textbook (p267).

Early Stopping is a form of Weight Decay

8111828

Because models typically start with relatively small weight values, **stopping gradient descent early** can be viewed as a form of **weight decay**.

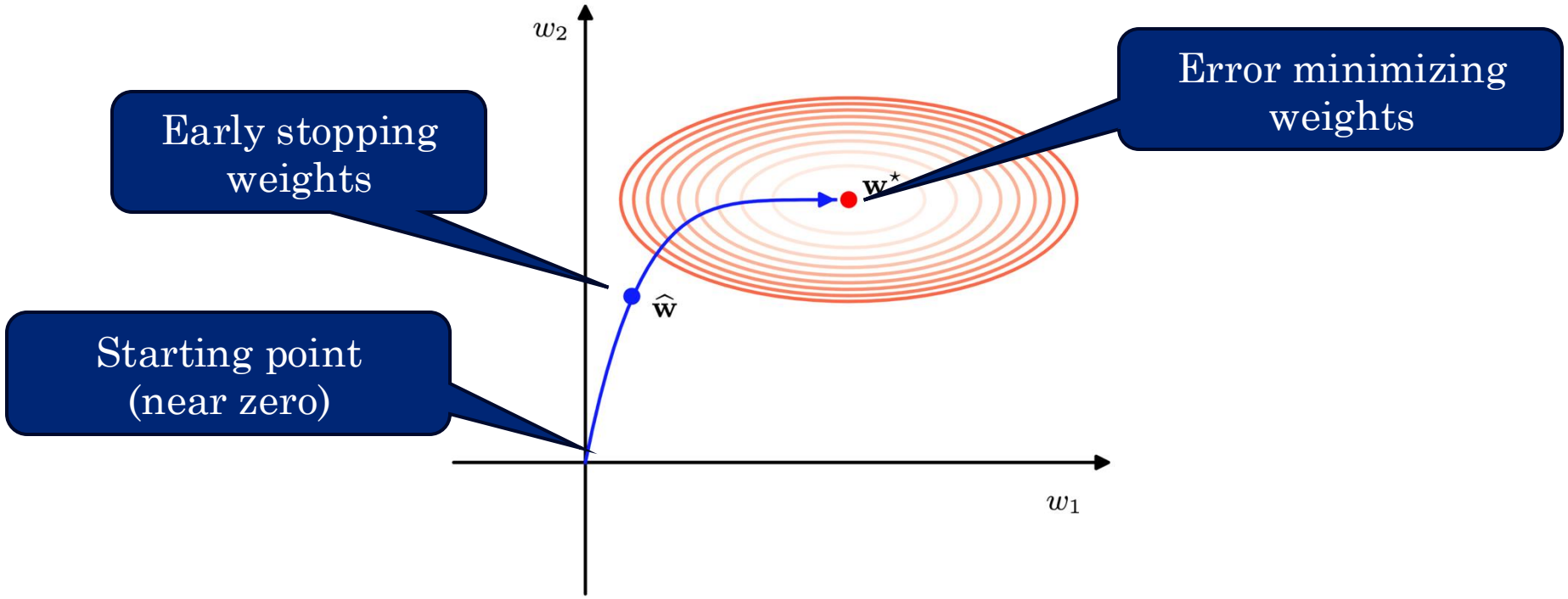


Figure from Bishop Textbook (p268).

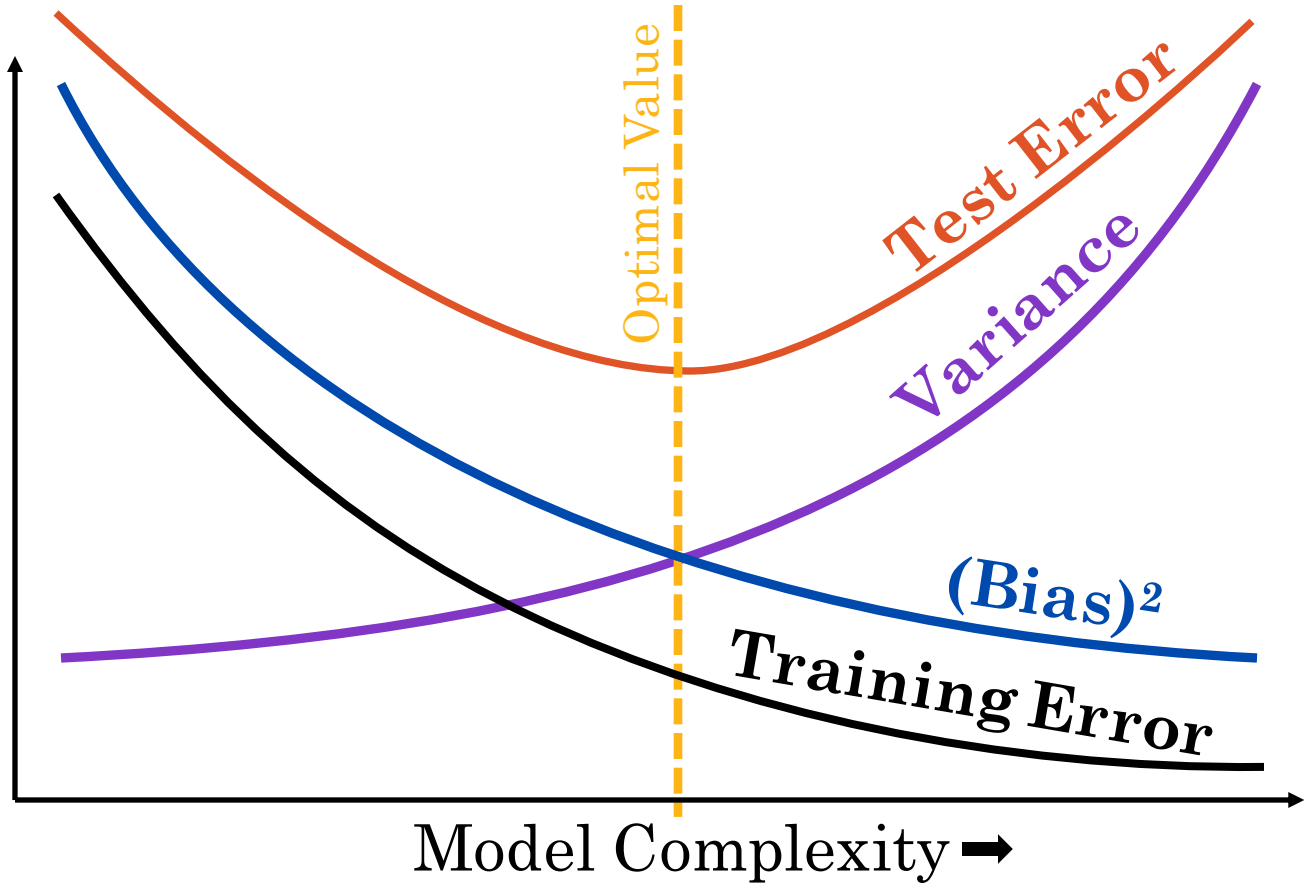
Bishop 9.3.1

Double Descent on the Bias-Variance Tradeoff

The Bias Variance Tradeoff Revisited

8111828

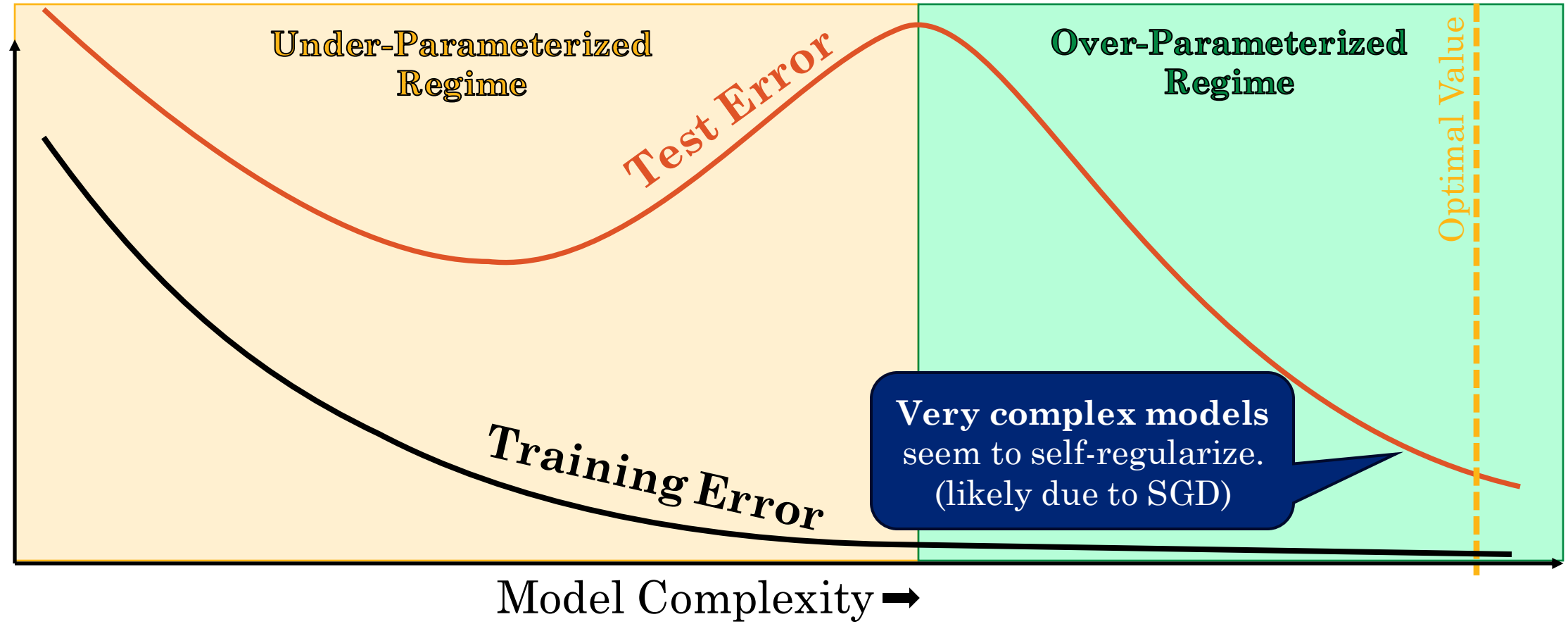
In previous lectures, we introduced the fundamental **bias-variance tradeoff** and how **bias** and **variance** contribute to **test-error**



Double Descent

8111828

In the case of **deep neural networks trained using SGD** that there is a **second decrease** in test error.

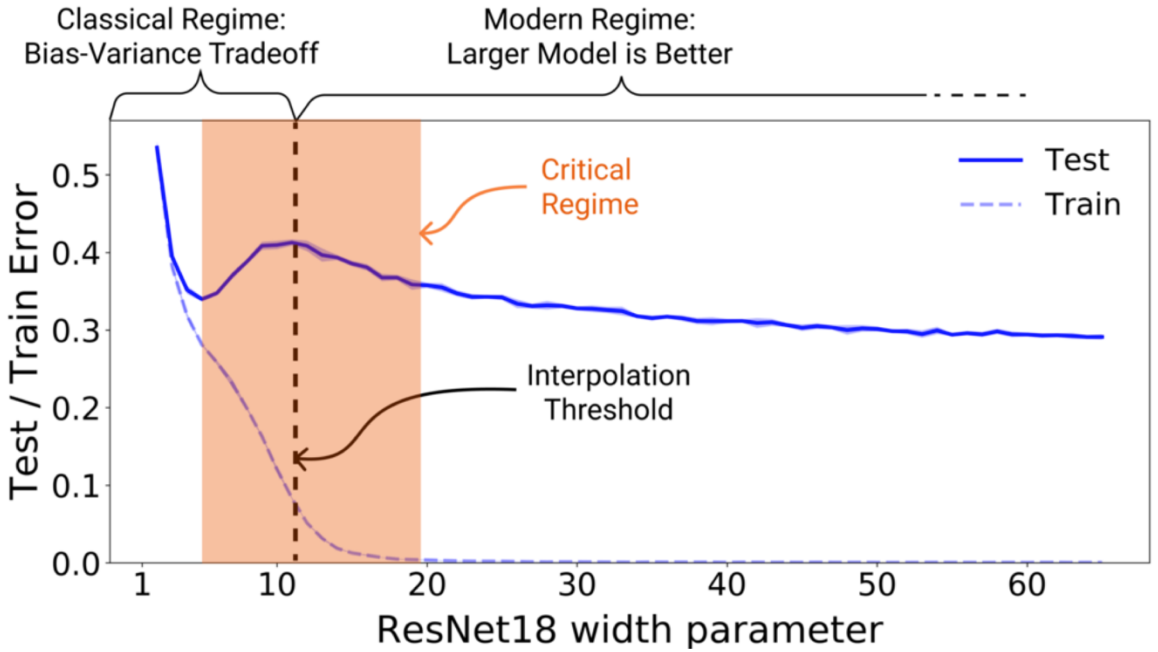


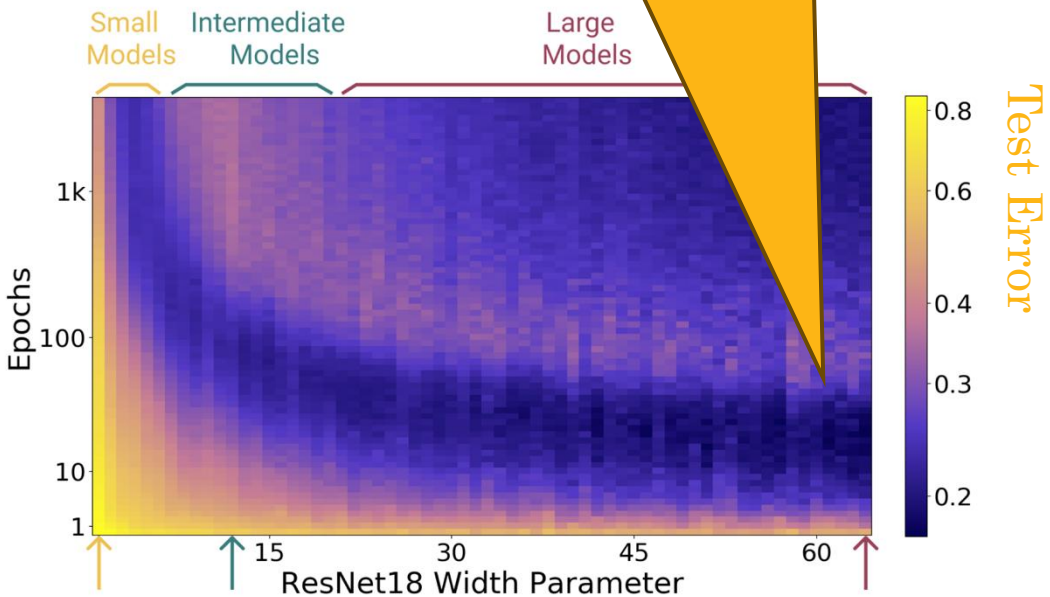
Double Descent in Practice

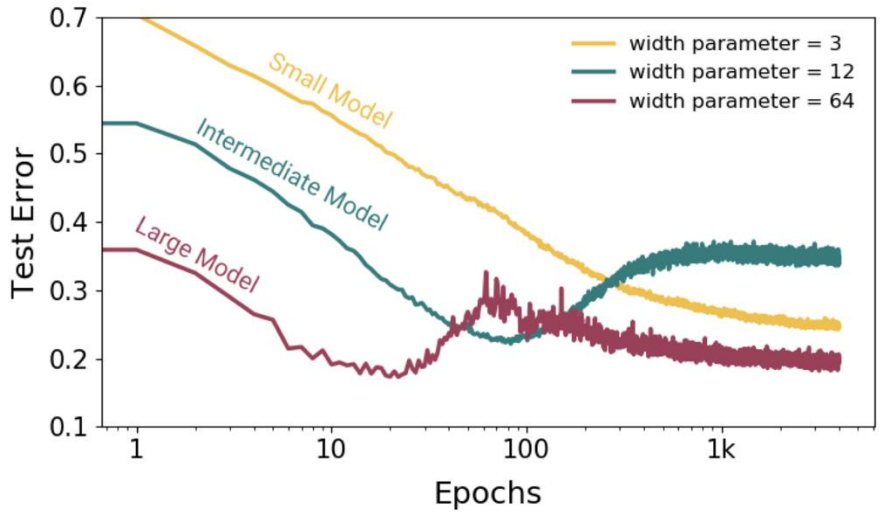
For sufficiently large models, early stopping could decrease generalization.

8111828

Examples of double descent from Nakkiran et al. ICLR'20:







Model Averaging

Ensembles of Models (Experts)

8111828

If you have K models $p_k(y \mid x)$ for a prediction task $p(y \mid x)$ then you can often construct an **ensemble model** that **combines the predictions** of each model to **improve accuracy**

$$p_{\text{ens}}(y \mid x) = \frac{1}{K} \sum_{k=1}^{K} p_k(y \mid x)$$

This works by reducing the **prediction variance**:

$$\text{Var}[p_{\text{ens}}(y \mid x)] = \frac{1}{K^2} \sum_{k=1}^{K} \text{Var}[p_k(y \mid x)] = \frac{1}{K^2} \sum_{k=1}^{K} \sigma^2 = \frac{\sigma^2}{K}$$

assuming the **IID predictions** with **variance** σ^2.

- Seldom true in practice

Mixture of Experts (MoEs)

8111828

A **mixture of experts** is a **generalization** of the **ensemble method** in which we assign learned weights $\pi(k \mid x)$ to each expert

$$p_{\text{MoE}}(y \mid x) = \sum_{k=1}^{K} \pi(k \mid x)\, p_k(y \mid x)$$

The **mixture weights** $\pi(k \mid x)$ form a probability distribution and often sparse ($\pi(k \mid x) = 0$ for many k).

- Sometimes called a **routing function**

Mixture of experts (MoEs) are often used as **internal layers** of neural networks as well. (more on this with LLMs)

Constructing Multiple Experts

8111828

Ideally, each expert is a **strong independent model**.

- **Strong:** State-of-the-art (**SoTA ←You should know this word!)**
- **Independent:** Trained with different data and inductive biases

Ensembles are could be constructed to by taking **SoTA** models from **competing teams** and **combining them**.

- Ensemble methods topped most classic benchmarks.

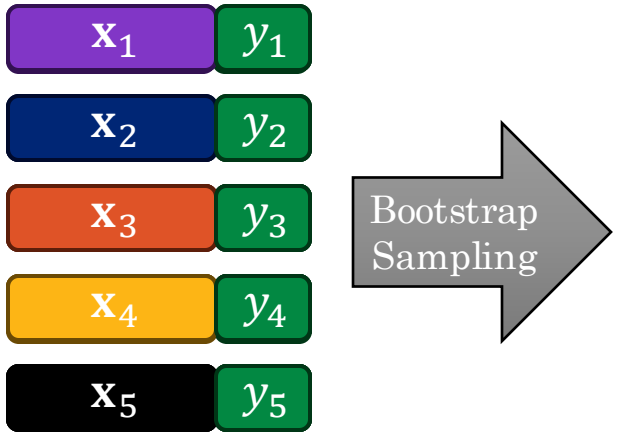
Bootstrap Aggregation (Bagging) is a classic technique to construct an ensemble by training on **bootstrap samp**les of the training data.

Bagging: Bootstrap Aggregation

8111828

Resample the training data K-times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.

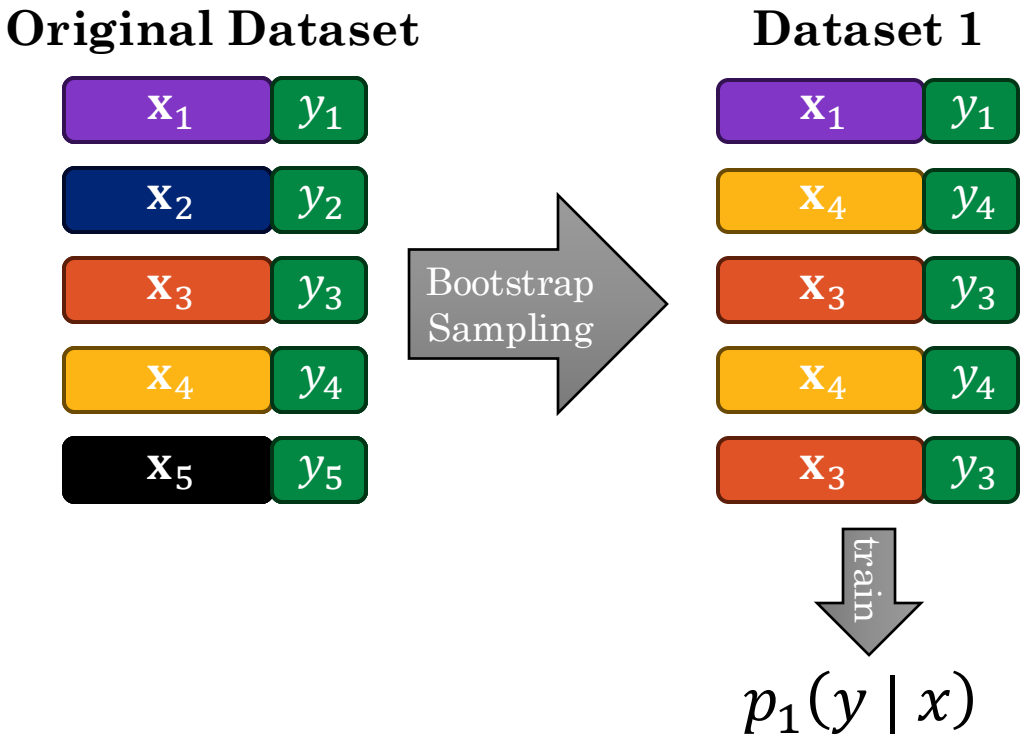
Original Dataset



Bagging: Bootstrap Aggregation

8111828

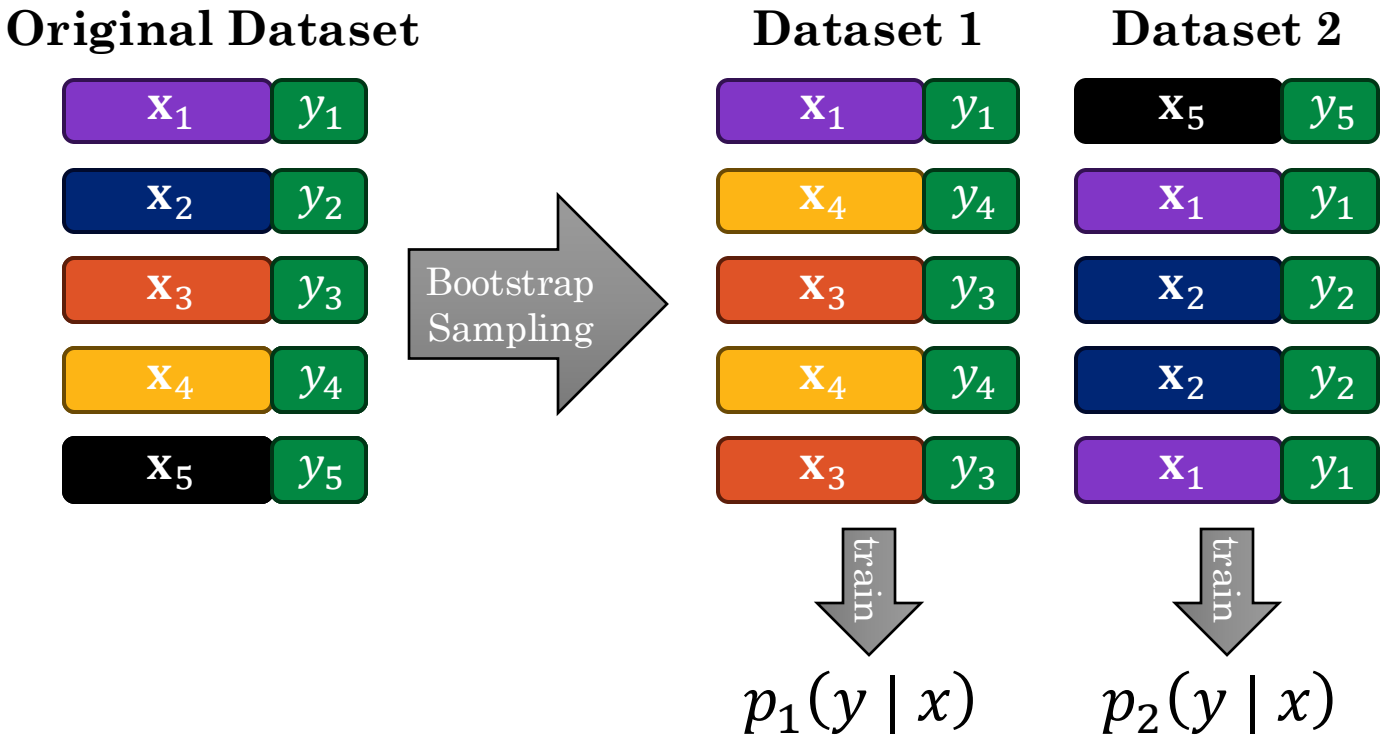
Resample the training data K-times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.



Bagging: Bootstrap Aggregation

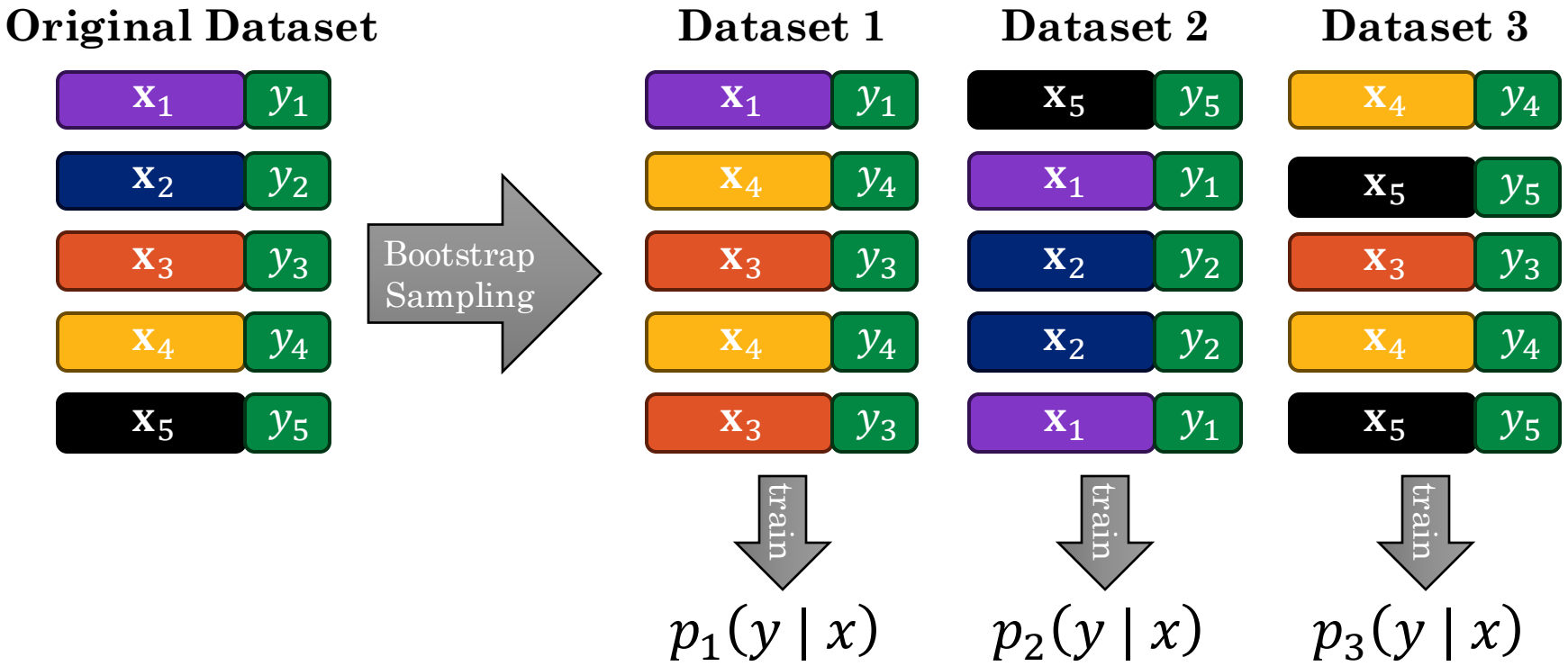
8111828

Resample the training data K-times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.



Bagging: Bootstrap Aggregation

Resample the training data K-times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.



Bagged Model:

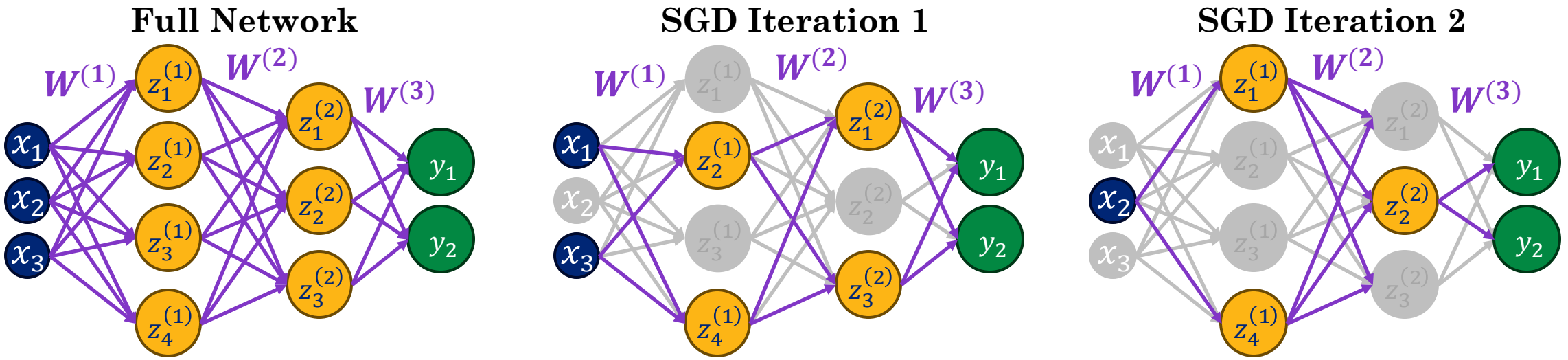
$$p_{\text{BAG}}(y \mid x) = \frac{1}{3}\sum_{k=1}^{3} p_k(y \mid x)$$

Dropout

Dropout Regularization

8111828

Dropout **regularizes neural networks** by **randomly "disabling" neurons** during each iteration of SGD.



Implemented by sampling $r_m^{(l)} \sim$ **Bernoulli**$(1-\rho)$ and then using $\tilde{z}_m^{(l)} = z_m^{(l)} * r_m^{(l)}$ as the modified activations (typically $\rho \leq .5$).

At **test time** we set $r_m^{(l)} = 1-\rho$ using all the activations (scaled).

What inductive bias is introduced by dropout?

Drop-out and Model Averaging

8111828

For a neural network with $\mathrm{M}_{\Sigma} = \mathrm{D} + \sum_{l=1}^{L} M_l$ total internal input neurons, drop-out **samples** from $\mathbf{2}^{\boldsymbol{M_{\Sigma}}}$ **possible "thinned" networks.**

During training, each step of SGD, we sample one of these thinned networks and apply a gradient updates to that network.

At test time, we are computing the **expected activation** by averaging over all possible combinations of input neurons.

Drop-out can be viewed as a form of **model averaging** over the $2^{M_{\Sigma}}$ thinned networks **with weight sharing.**

Implementing Drop-out in PyTorch

8111828

```
dropout = nn.Dropout(p=0.3)

dropout.train()
display(dropout(torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]])))

dropout.eval()
display(dropout(torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]])))
```

✓ 0.0s

```
tensor([[ 1.4286,  2.8571,  0.0000],
        [ 5.7143,  7.1429,  0.0000],
        [10.0000, 11.4286, 12.8571]])

tensor([[1., 2., 3.],
        [4., 5., 6.],
        [7., 8., 9.]])
```

Residual Connections

Vanishing Gradients in Deep Networks

8111828

Training networks with hundreds of layers is challenging.

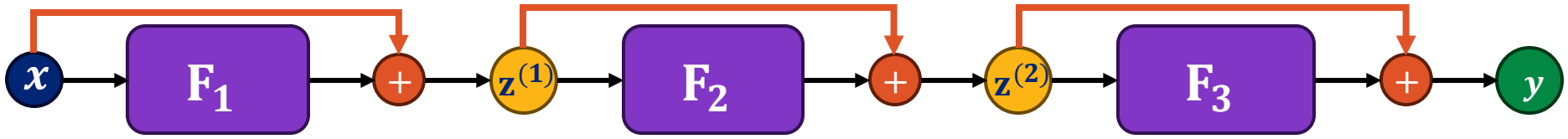
- **Vanishing Gradients:** error signals get multiplicative diluted across layers
- **Shattered Gradients:** many stacked non-linearities result in a rough loss surface.

Needed a way to **connect lower layers** in the network directly to the predictions (and subsequent loss).

Residual Networks

8111828

Residual networks enable the training of very deep networks (e.g., hundreds of layers) by introducing **residual connections** (skip connections) **around blocks** (sub-networks F_l).



We can express a residual block as:

$$\mathbf{z}^{(l)} = \mathrm{F}_l\left(\mathbf{z}^{(l-1)}\right) + \mathbf{z}^{(l-1)}$$

Example:
$\mathrm{F}_l(\mathbf{z}) = W^{(l)}\,\mathbf{ReLU}(\mathbf{z})$

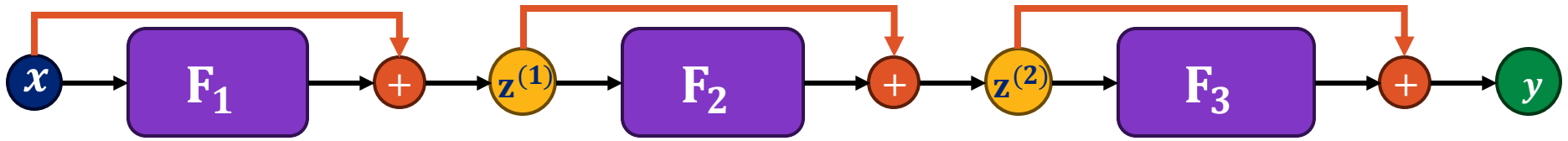
The term **residual** comes from the fact that F_l is modeling the **residual** between the **output** and the **input** of the layer:

$$\mathrm{F}_l\left(\mathbf{z}^{(l-1)}\right) = \mathbf{z}^{(l)} - \mathbf{z}^{(l-1)}$$

Residual Networks as Ensembles

8111828

Residual networks can be viewed as an **ensemble of networks with varying depths:**



If we expand our example network to get:

$$y = \mathrm{F}_3\big(\mathrm{F}_2(\mathrm{F}_1(\mathbf{x}) + \mathbf{x}) + \mathbf{z}^{(1)}\big) + \mathbf{z}^{(2)}$$

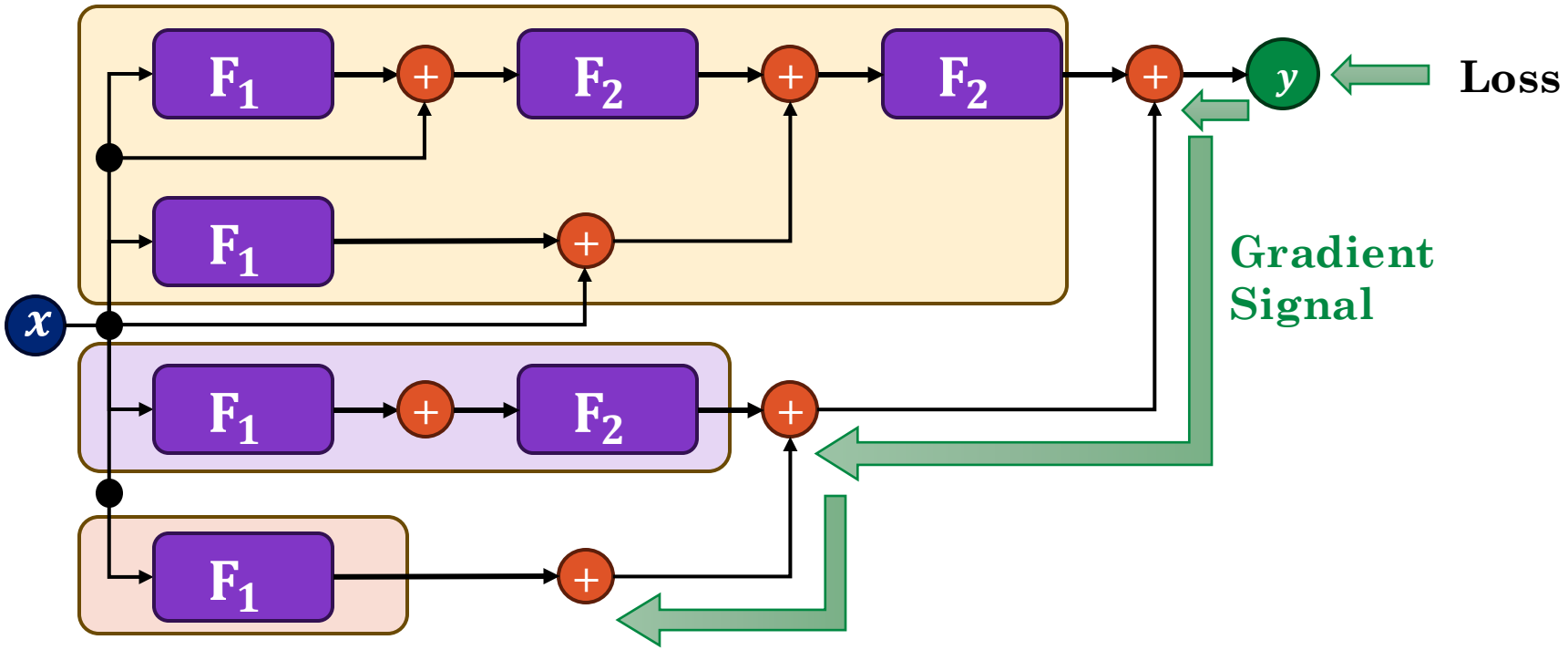
Substituting the $\mathbf{z}^{(1)} = \mathrm{F}_1(\mathbf{x}) + \mathbf{x}$ and $\mathbf{z}^{(2)} = \mathrm{F}_2(\mathrm{F}_1(\mathbf{x}) + \mathbf{x}) + \mathrm{F}_1(\mathbf{x}) + \mathbf{x}$:

$$y = \underbrace{\mathrm{F}_3(\mathrm{F}_2(\mathrm{F}_1(\mathbf{x}) + \mathbf{x}) + \mathrm{F}_1(\mathbf{x}) + \mathbf{x})}_{\textbf{Large Network}} + \underbrace{\mathrm{F}_2(\mathrm{F}_1(\mathbf{x}) + \mathbf{x})}_{\textbf{Medium Network}} + \underbrace{\mathrm{F}_1(\mathbf{x}) + \mathbf{x}}_{\textbf{Small Network}}$$

Expanding the Residual Network Graph

The **expanded ensemble structure** of the residual network shows how **input layers** receive more **direct gradient signal**.

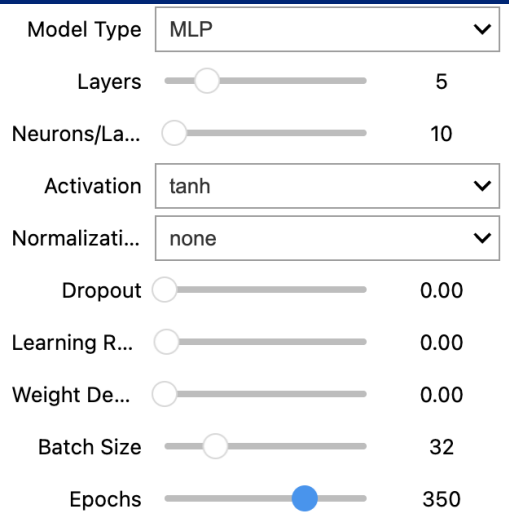
$$y = \mathrm{F}_3(\mathrm{F}_2(\mathrm{F}_1(\mathbf{x}) + \mathbf{x}) + \mathrm{F}_1(\mathbf{x}) + \mathbf{x}) + \mathrm{F}_2(\mathrm{F}_1(\mathbf{x}) + \mathbf{x}) + \mathrm{F}_1(\mathbf{x}) + \mathbf{x}$$

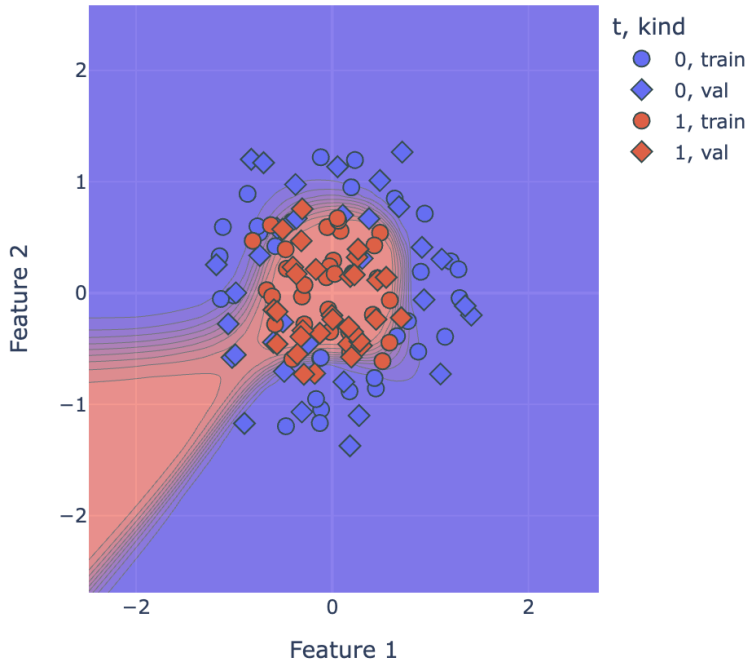


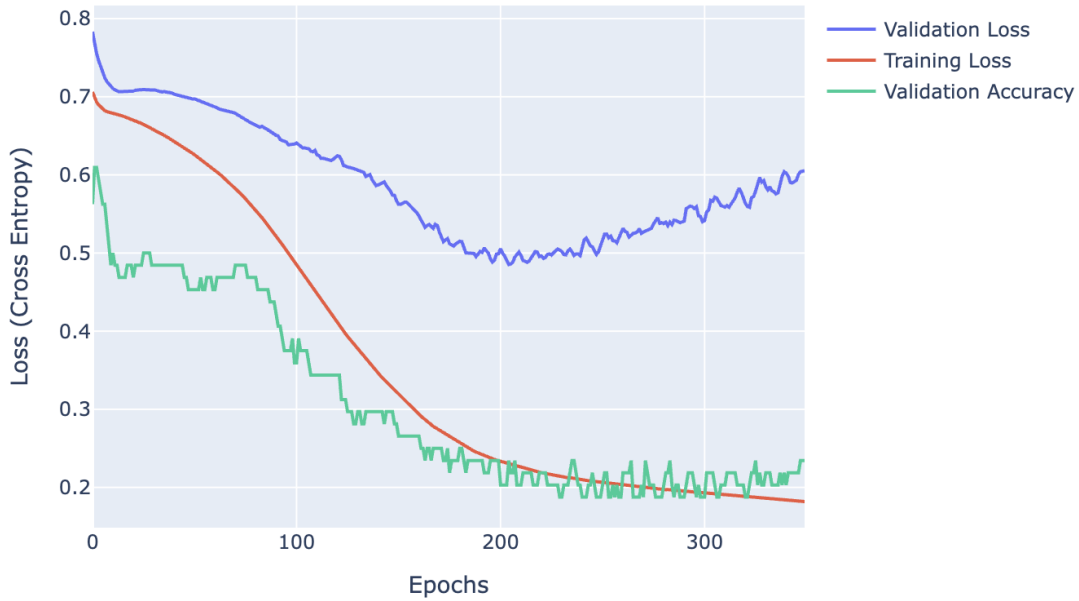
8111828

Demo

Probably the coolest demo yet.







Lecture 16

Neural Network Design and Regularization

Reading: *Chapter 8* in Bishop Deep Learning Textbook